Curs 4

Calea de date

Calea de date (data path, eng.) a unui procesor DSP este locul din structura unde au loc operatiile de prelucrare a datelor. La procesoarele DSP calea de date este foarte specializata pentru a atinge performante inalte la executia operatiilor cerute de algoritmii DSP, cum ar fi operatiile de transfer de date, de multiplicare si acumulare, scalare, s.a.. Facilitatile caii de date, impreuna cu arhitectura memoriei (analizata in capitolul 5), sunt caracteristicile care diferentiaza cel mai clar procesoarele DSP de alte tipuri de procesoare. Calea de date la procesoarele DSP in virgula mobila este semnificativ diferita de cea de la procesoarele DSP in virgula fixa datorita cerintelor diferite ale aritmeticilor folosite. Vom analiza mai intai structura caii de date pentru procesoarele cu virgula fixa si apoi pentru procesoarele cele cu virgula mobila.

4.1 Calea de date la procesoarele cu virgula fixa

Calea de date la procesoarele DSP cu virgula fixa contine: multiplicator, ALU (unitate logica-aritmetica), unul sau mai multe registre pentru deplasatori (shifters), registre de lucru, acumulatoare, precum si alte unitati specializate. Unii producatori denumesc intreaga cale de date ALU sau unitate aritmetica. In continuare vom folosi termenul ALU pentru combinatia de unitati functionale sumator/scazator/partea logica a caii de date. Denumirea de “cale de date” este folosita pentru traseul complet de procesare, incluzind multiplicator, acumulatori, s.a.m.d.. Figura 4-1 prezinta calea de date pentru un procesorul cu aritmetica cu virgula fixa pe 24 de biti, DSP5600x (Motorola).

In general, calea de date la procesoarele DSP nu este folosita si pentru efectuarea calculelor legate de adresarea memoriei. In schimb, procesoarele DSP cu virgula fixa poseda o unitate hardware separata pentru calculul adresei, denumita unitate de generare a adresei (address generation unit). Acest modul executa o varietate de calcule pentru adresare, cum ar fi adresarea modulo, adresarea cu inversarea bitilor, etc.. Gama larga de moduri de adresare specializate existente la procesoarele DSP reprezinta unul dintre factorii care deosebesc procesoarele DSP de alte procesoare, cum ar fi procesoarele de uz general. Unitatile de generare a adresei ale si modurile de adresare sunt discutate in detaliu in Capitolul 6. Vom analiza in continuare principalele unitati ale caii de date.

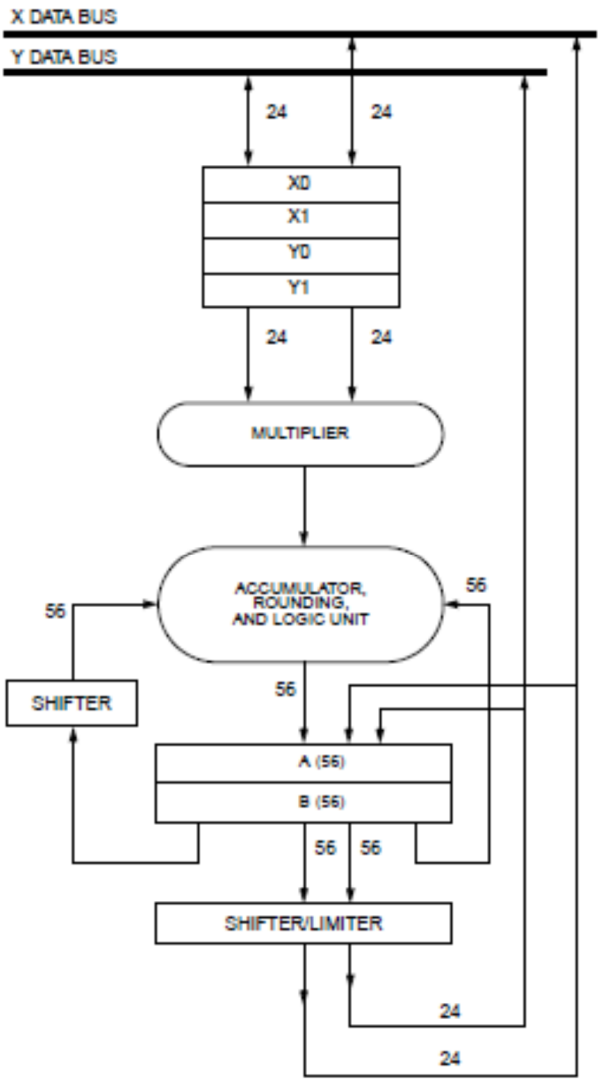


Fig. 4.1 Calea de date la procesorul cu virgula fixa pe 24 de biti DSP 5600x (Motorola)

- **Multiplicatorul**

Multiplicarea este o operatie esentiala in toate aplicatiile DSP. In multe situatii, jumatate sau chiar mai multe operatii executate de procesor implica multiplicarea. Din acest motiv aproape toate procesoarele DSP contin un multiplicator hardware de tip paralel care poate inmulti doi operanzi de marime nativa intr-un singur ciclu instructiune. Exista insa diferente importante in ceea ce priveste modul de efectuare a operatiei de multiplicare intre diferite procesoare DSP.

Cu toate ca procesoarele DSP sunt echipate cu un multiplicator care produce un nou rezultat la fiecare ciclu instructiune, pipeline-ul intern al multiplicatorului poate duce la o intarziere mai mare de un ciclu de la aplicarea operanzilor la intrarile multiplicatorului si pana cand este disponibil rezultatul. Aceasta intarziere este numita timp de latenta. Multiplicatoarele cu pipeline furnizeaza un rezultat la fiecare ciclu de tact doar atunci cand sunt executate siruri lungi de operatii de multiplicare. Daca sunt executate inmultiri izolate, urmate de alte operatii, rezultatele sunt disponibile dupa unul sau mai multe cicluri de tact. Un exemplu de procesor DSP cu multiplicator pipeline este nucleul DSP CD2450 (Clarkspur Design).

In unele cazuri (DSP5600x - Motorola) multiplicatorul este integrat cu un sumator formind o unitate multiplicator-acumulator. In alte cazuri (DSP16xx - AT&T) multiplicatorul este separat; iesirea este stocata intr-un registru de produs, si de acolo poate fi trimis la un sumator pentru adunare. Aceasta diferenta se observa in cresterea timpului de latenta a operatiei de multiplicare-acumulare. Daca multiplicarea si adunarea sunt separate, rezultatul operatiei de multiplicare-acumulare este in general intarziat cu un ciclu instructiune inainte de a putea fi folosit de instructiunea urmatoare.

Alta diferenta intre multiplicatoarele procesoarelor DSP cu virgula fixa se refera la marimea produsului fata de marimea operanzilor. Intotdeauna cand inmultim doua numere in virgula fixa de marime *n* biti, sunt necesari 2 *n* biti pentru a reprezenta produsul rezultat fara a introduce o eroare. Aceasta proprietate este asa-numita legea de conservare a bitilor.

Pentru a intelege de ce se intampla asa, sa consideram reprezentarea intregilor pe 8 biti, prezentata in figura 3-2 din capitolul 3. Acest format este capabil sa reprezinte numere intre –128 si +127. Daca inmultim doua numere pe 8 biti de amplitudine mare rezultatul devine prea mare pentru a fi reprezentat intr-un format pe 8 biti. De exemplu, 128 x 128 = 16.384; pentru a reprezenta acest rezultat, avem nevoie de un format de reprezentare de 16 biti.

Prin urmare operatia de multiplicare efectuata de procesoarele DSP cu virgula fixa produc un rezultat de doua ori mai mare fata de marimea operanzilor de intrare. Daca pot fi reprezentati toti bitii rezultatului multiplicarea nu introduce erori de calcul. Totusi unele procesoare cu virgula fixa, din motive de viteza si economie, folosesc multiplicatoare care produc rezultate cu un numar de biti mai redus si astfel introduc erori in unele calcule. De exemplu, la procesorul Zilog - Z893xx (si la nucleul Clarkspur CD2400, pe care este bazat Z893xx), multiplicatorul accepta operanzi pe 16 biti, dar produce un rezultat pe 24 biti si nu unul pe 32 de biti cum ar fi necesar pentru functionarea cu precizie maxima.

Desi este posibila folosirea rezultatului multiplicarii nemodificat la urmatorul pas de calcul, in general nu se procedeaza asa deoarece operatii ulterioare de multiplicare sau adunare ar produce rezultate si mai lungi. Totusi, in majoritatea cazurilor nu este necesar sa retinem rezultatul intreg al multiplicarii, deoarece exista adesea un punct dincolo de care cresterea gamei dinamice si a preciziei nu mai prezinta interes. De aceea, programatorul selecteaza un subset din bitii de la iesirea multiplicatorului care sunt folositi la urmatorul pas de calcul. Sau, daca un sir de produse urmeaza sa fie adunate, adunarea poate fi facuta pastrind toti bitii produselor, si reducind in final lungimea datei rezultate, inainte de a trece la urmatoarea etapa de calcul.

Daca nu trebuie pastrati toti bitii rezultatului multiplicarii sau adunarii, programatorul trebuie sa selecteze partea rezultatului care sa fie retinuta. Aceasta selectie este facuta astfel incat virgula portiunii de rezultat selectata sa fie in aceeasi pozitie in care este virgula in formatul cu

virgula fixa folosit peste tot in program. In acest fel iesirea multiplicatorului poate fi tratata ca orice alt operand in decursul prelucrarii.

Cand doua numere binare sunt inmultite, pozitia virgulei in rezultatul cu lungime completa depinde de pozitia virgulei operanzilor. Aceast lucru este ilustrat in Fig. 4.2 si 4.3. Daca este folosita aritmatica intreaga, iesirea nemodificata a multiplicatorului este de asemenea un intreg, si programatorul retine in general cei mai putin semnificativi biti ai rezultatului. In acest caz, marimea operanzilor trebuie constransa de operator astfel incat valoarea semnificativa a rezultatului multiplicarii sa ocupe jumatatea de ordin inferior a cuvintului care va fi selectat pentru urmatorul pas. Cand acest lucru este realizat, cei mai semnificativi $n/2+1$ biti din cei n biti ai rezultatului sunt egali cu valoarea bitului de semn (uneori sunt numiti biti de extensie a semnului). Astfel nu este pierduta nici o informatie cand cei $n/2$ biti de ordin superior sunt eliminati (pastrind un singur bit de semn in cuvantul final de la iesire).

Daca este folosita aritmetica fractionara, rezultatul cu precizie completa al multiplicarii are de doua ori mai multi biti la dreapta virgulei decat operanzii multiplicarii, iar programatorul descarca in general jumatatea cea mai putin semnificativa a acestor biti, posibil prin rotunjire.

Deosebirea dintre care parte a rezultatului multiplicarii este selectata pentru a fi folosita in urmatorul pas al calculelor constituie diferenta principala intre aritmetica intreaga si cea fractionara la procesoarele DSP cu virgula fixa.

O optimizare obisnuita folosita la efectuarea multiplicarii in aritmetica cu virgula fixa consta in a interzice unuia din operanzi de a lua cea mai negativa valoare care poate fi reprezentata cu formatul numeric folosit (aceasta valoare este uneori numita NMAX). Daca acesta optimizare este facuta, marimea rezultatului nemodificat este redusa cu un bit. Cand doua numere binare pe n biti sunt inmultite, daca cel putin unul nu este cel mai negativ numar reprezentabil, largimea in biti a rezultatului este $2n$-1 biti, in loc de $2n$ biti. Asa cum este aratat in figura 4-3, cand este folosita aritmetica fractionara, aceasta optimizare conduce la un cuvant de iesire care contine un singur bit de semn si nici un bit intreg la stanga virgulei. Aceasta permite programatorului sa selecteze un subset din rezultatul nemodificat care produce un cuvant la iesire cu acelasi format ca si operanzii de intrare. Fara acesta optimizare, ar exista un singur bit de semn plus un singur bit intreg la stanga virgulei in rezultatul nemodificat. Programatorul ar trebui atunci sa scaleze rezultatul nemodificat inainte sa selecteze un cuvant de iesire cu acelasi format cu operanzii de intrare. Procesorul poate efectua aceasta scalare fara instructiuni suplimentare, scalarea avand efectul de reducere a gamei dinamice a datelor.

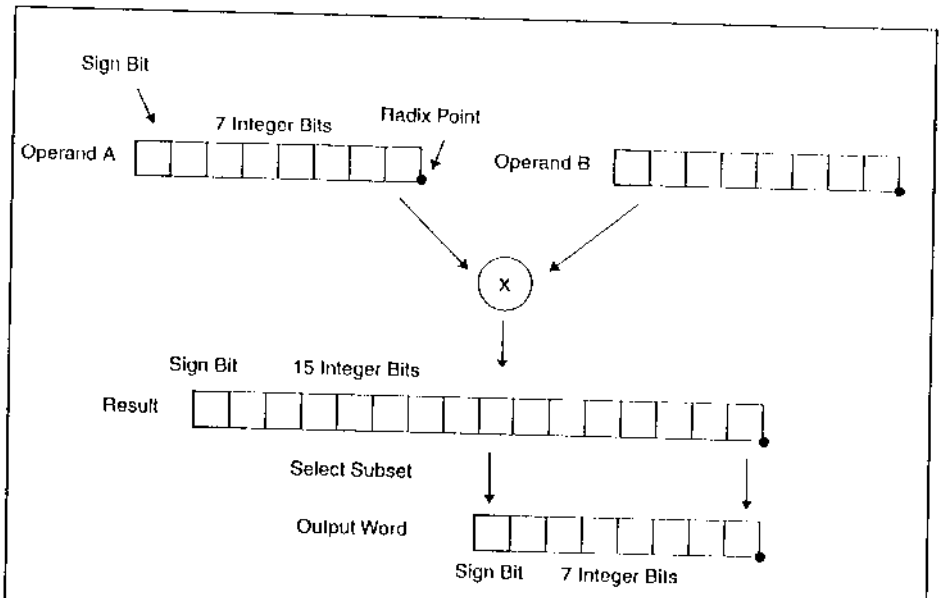


FIGURE 4-2. When multiplying integer values, the result is also an integer, with twice the overall width of the input operands. Normally, the programmer selects the least significant half of the result word for use in the next step of processing. In this case, the programmer must ensure that the full result value is contained within the lower half of the result word to prevent overflow. This can be done by constraining the size of the operands or by scaling the result. When the operands are constrained or the results have been scaled properly, the most significant *N*/2+1 bits of the *N*-bit result are all equal to the value of the sign bit, so no information is lost when the upper *N*/2 bits are discarded (leaving one sign bit in the final output word).

Fig. 4.2

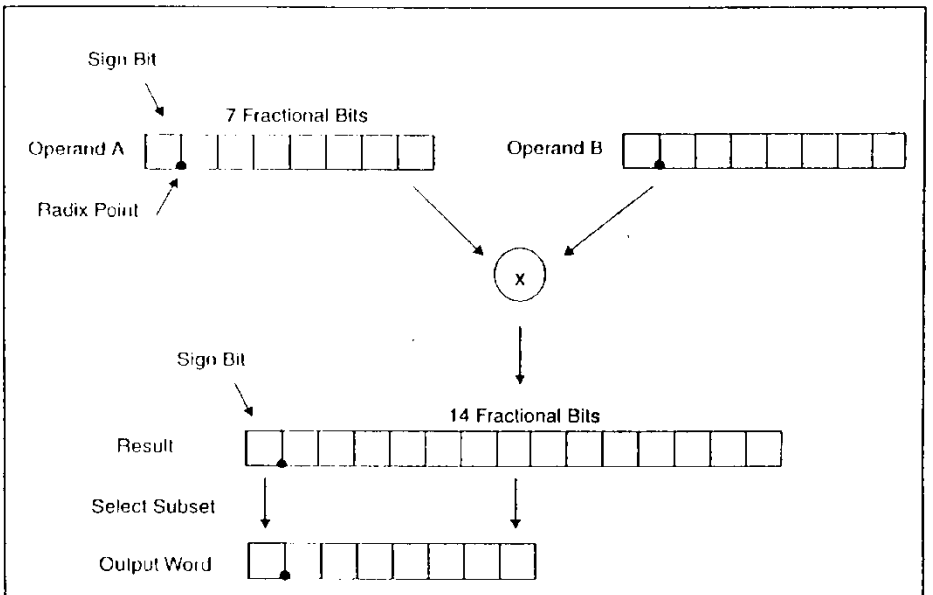


FIGURE 4-3. When multiplying fractional values, the result is also a fractional value. The result has a number of bits to the right of the radix point which is equal to the sum of the number of bits to the right of the radix point in the two operands. If the operands are constrained so that at least one operand is not the largest negative number representable, then the bit to the left of the radix point is the sign bit. This means that the programmer can select the subset of the result shown above, yielding an output value with the same fractional format as the input operands. Without this optimization, there would be a single sign bit plus a single integer bit to the left of the radix point in the full-width result.

Adesea procesarele DSP cu virgula fixa poseda hardware specific care ajuta programatorul sa selecteze eficient subsetul de biti de la iesirea multiplicatorului. O caracteristica obisnuita consta in a considera registrul de la iesirea multiplicatorului (de lungime $2n$ biti) sau registrul acumulator, ca doi registri de n biti, adresabili independent. Aceasta permite programatorului sa selecteze direct cea mai semnificativa jumatate (pentru multiplicarea fractiilor) sau cea mai putin semnificativa jumatate (pentru multiplicarea intregilor) a cuvintului de la iesirea multiplicatorului sau din accumulator pentru a fi folosita la urmatorul pas de calcul. Alta particularitate obisnuita consta in executarea unei deplasari automate la stanga cu un bit dupa multiplicare. In cazul in care optimizarea largimii cuvintelor, discutata mai sus, este folosita pentru multiplicarea fractiilor, operatia de deplasare la stanga aliniaza iesirea multiplicatorului astfel incat subsetul dorit al rezultatului nemodificat sa se alinieza cu portiunea mai semnificativa a registrului rezultatului. Daca jumatatea mai semnificativa a registrului rezultatului este adresabila independent, atunci programatorul poate selecta bitii doriti din rezultatul nemodificat citind direct jumatatea semnificativa a acestui registru. Fara aceasta deplasare, programatorul ar trebui sa execute explicit o operatie de deplasare inainte de a citi portiunea dorita din rezultat. La unele procesoare, acesta deplasare la stanga poate fi activata sau dezactivata prin folosirea unui bit de mod, simplificind trecerea intre aritmetica fractionara si cea cu intreagi.

- **Registrii acumulator**

Registrii acumulator retin rezultate partiale si finale ale operatiei de multiplicare-acumulare si ale altor operatii aritmetice. Majoritatea procesoarelor DSP poseda doi sau mai multi acumulatori. Cateva procesoare au insa un singur acumulator, ceea ce poate reprezenta o limitare pentru multe aplicatii. Cand este disponibil doar un acumulator, acesta devine deseori o “gituire” in arhitectura; deoarece acumulatorul este in general folosit pentru unul din operanzii sursa si ca operand destinatie pentru operatiile executate de ALU, continutul sau trebuie sa fie incarcat sau stocat frecvent intrucit ALU este folosit si pentru alte scopuri. Aceste transferuri limiteaza rata la care datele pot trece prin ALU.

In mod ideal, pentru a pastra precizia de reprezentare, marimea registrilor acumulator ar trebui sa fie mai mare decat marimea cuvantului de la iesirea multiplicatorului cu cativa biti. Bitii suplimantari, numiti *biti de garda*, permit programatorului sa adune un numar de valori fara riscul aparitiei depasirii de scala in acumulator si fara sa fie necesara scalarea rezultatelor intremediare pentru a evita depasirea scalei (*overflow*). Un acumulator cu n biti de garda face posibila adunarea a pana la 2^n valori fara sa apara depasirea scalei. Majoritatea procesoarelor furnizeaza fie patru, fie opt biti de garda. De exemplu, procesoarele DSP16xx (AT&T) asigura patru biti de garda (acumulatoarele au capacitatea de 36 biti, iar multiplicatorul lucreaza pe 32 biti), in timp ce ADSP-

21xx (Analog Devices) asigura opt biti de garda (acumulatoare pe 40 biti cu multiplicatoare pe 32 biti).

Intr-un procesor fara biti de garda, datele de intrare sau rezultatele intermediare trebuie adesea sa fie scalate inainte sa fie adunate in acumulator, daca dorim sa eliminam posibilitatea aparitiei depasirii. In general aceasta implica scalarea rezultatului inmultirii prin deplasarea sa la dreapta cu cativa biti. Unele procesoare care nu poseda biti de garda in acumulator sunt capabile sa deplaseze valoarea registrului produsului inainte de a o aduna la acumulator fara ca aceasta sa implice cicluri instructiune suplimentare. De exemplu, TMS320C2x si TMS320C5x (Texas Instruments) permit deplasarea automata la dreapta cu sase biti a continutului registrului de produs. Asa cum este aratat in continuare o asemenea scalare duce la o pierdere a preciziei. Oricum, cu exceptia cazului in care marimea scalarii folosite este extrema sau numarul de produse acumulate este foarte mare, pierderea de precizie introdusa de operatia de scalare este mica.

Un alt argument in favoarea deplasarii produsului la dreapta inaintea adunarii este faptul ca atunci cand este folosita aritmetica fractionara, deseori doar cea mai importanta jumatate a rezultatului adunarii este retinuta dupa o serie de operatii de multiplicare-adunare. In acest caz, pierderea preciziei valorilor intermediare datorata scalarii nu afecteaza rezultatul final. Aceasta se intampla deoarece erorea de cuantizare prezenta in rezultat din cauza scalarii este continuta in totalitate in jumatatea mai putin semnificativa a acumulatorului care este descarcata.

Bitii de garda asigura o mai mare flexibilitate in evitarea depasirilor de scala decat scalarea produsului multiplicarii deoarece permit ca precizia maxima sa fie mentinuta in etapele intermediare ale calculului. Totusi, posibilitatea de scalare a rezultatului multiplicarii in lipsa bitilor de garda este suficienta pentru multe aplicatii. Cateva procesoare, cum ar fi TMS320C1x (Texas Instruments), sunt lipsite de ambele posibilitati. Aceasta impune ca iesirea multiplicatorului sa fie scalata pentru a evita depasirea, ceea ce poate duce la o reducare semnificativa a preciziei. Lipsa, atat a bitilor de garda, cat si a suportului pentru scalarea continutului registrului produs poate fi o limitare serioasa in multe situatii.

- **ALU**

Unitatile logice aritmetice ale procesoarelor DSP executa operatiile logice si aritmetice de baza intr-un singur ciclu instructiune. Operatiile obisnuite includ adunarea, scaderea, incrementarea, negarea, si operatiile logice SI, SAU, NU. Unitatile ALU difera intre ele prin marimea cuvintelor folosite pentru operatii logice. Unele procesoare realizeaza operatii logice asupra operanzilor care au largimea completa a acumulatorului, in timp ce altele pot face operatii logice numai asupra cuvintelor de date de largime nativa. De exemplu, DSP16xx (AT&T) realizeaza operatii logice asupra tuturor bitilor acumulatorului pe 36 biti, in timp ce DSP5600x (Motorola), care are un acumulator pe 56 biti, realizeaza operatii logice doar asupra cuvintelor de date native pe 24 biti.

Daca ALU nu poate executa operatii logice simultan asupra datelor de lungimea acumulatorului, programatorii care au nevoie de aceasta facilitate trebuie sa execute operatiile logice in mai multe etape, ceea ce complica programarea si consuma suplimentar cicluri instructiune.

Asa cum am mentionat, in unele procesoare ALU este folosita pentru a executa adunarea pentru operatia de multiplicare-acumulare. In alte procesoare, este executata in acest scop o adunare separata.

- **Deplasatorul (shifter)**

Multiplicarea si acumularea duc la cresterea numarului de biti a rezultatelor operatiilor aritmetice. In majoritatea cazurilor, programatorul va alege un set particular de biti ai rezultatului pe care sa-l foloseasca la urmatoarea etapa de calcul. Un deplasator (*shifter*) pe calea de date usureaza aceasta selectie scalind (multiplicind) iesirea cu o data putere a lui doi.

Scalarea este o operatie importanta in aplicatii DSP implementate cu aritmetica cu virgula fixa deoarece multe operatii produc expandarea sau contractarea gamei valorilor semnalelor pe care le proceseaza.

Scalarea trebuie facuta cu mare grija, punind in balanta nevoia reduceri sau eliminarii posibilitatii de aparitie a depasirii scalei, cu nevoia mentinerii unei game dinamice si a unei precizii adecvate. Scalarea corecta a datelor poate reprezenta un punct sensibil la implementarea unei aplicatii folosind un procesor cu virgula fixa. Tipic, proiectantii de aplicatii folosesc simulari si analize detaliate pentru a determina unde si cu cat ar trebui sa fie scalate semnalele, iar apoi introduc operatii de scalare in locul potrivit.

Se observa ca scalarea este complementara folosirii bitilor de garda ai acumulatorului deoarece ambele tehnici sunt folosite pentru a elimina sau reduce posibilitatea de aparitie a depasirii scalei. In timp ce scalarea reduce gama rezultatelor intermediare sau finale la domeniul valorilor reprezentabile, bitii de garda asigura o reprezentare intermediara care are o gama dinamica mai larga. Asa cum am prezentat, scalarea rezultatelor intermediare poate fi uneori folosita pentru a inlatura nevoia folosirii bitilor de garda. In orice caz, bitii de garda nu inlatura nevoia scalarii. Cand bitii de garda sunt folositi este necesara scalarea rezultatului final pentru a face conversia de la reprezentarea intermediara la cea finala. De exemplu, un procesor pe 16 biti cu 4 biti de garda in acumulator, poate necesita scalarea valorilor acumulatorului cu 2^{-4} inainte de a le scrie in memorie ca valori pe 16 biti.

Adesea este prevazut un registru de deplasare (shifter) imediat dupa multiplicator si ALU. Unele procesoare poseda un deplasator intre multiplicator si ALU care permite scalarea produsului. Unele deplasatoare au capacitati limitate, de exemplu, executa deplasari la stanga cu un bit (scalare cu 2^{1}), deplasarea la drepta cu un bit (scalare cu 2^{-1}), sau fara deplasare. Exista si deplasatoare care pot executa si deplasari multibit, de tip bit cu bit, dar aceasta operatie poate consuma prea mult

timp. Unele procesoare folosesc un deplasator special, denumit *barrel shifter*, care ofera o mai mare flexibilitate putind realiza deplasari cu mai multe ranguri binare intr-un singur ciclu de tact. La unele procesoare, un deplasator de capacitate limitata este localizat imediat dupa multiplicator, astfel incat rezultatele multiplicarii pot fi scalate inainte sa fie transferate in acumulator sau ALU pentru a continua procesarea.

Exista procesoare DSP care poseda mai multe shiftere plasate in diferite locuri ale caii datelor. Aceasta permite o flexibilitate mai mare, putind executa deplasarea in locul potrivit, pentru fiecare algoritm care este implementat. De exemplu, DSP5600x are doua deplasatoare de capacitate limitata, independente: unul este folosit pentru a scala rezultatele operatiei de multiplicare-acumulare, pentru a fi transferate de la acumulator in memorie, si altul este folosit pentru o deplasa valorile intr-un acumulator, inclusiv pentru a efectua deplasari logice (fara extensie de semn) si rotatii.

- **Depasirea si saturatia**

Multe aplicatii DSP implica acumularea (adunarea unui sir de date). Aceasta se intampla, de exemplu, in algoritmii de filtrare, unde elementele unui sir de date sunt multiplicate cu coeficienti, iar produsele partiale sunt adunate. Cand un sir de numere este adunat, numarul de biti ai sumei poate creste. Ca urmare, suma poate depasi valoarea maxima care poate fi reprezentata in registrul acumulator. In aceasta situatie, numita *depasire de scala* (overflow), este inmagazinata o valoare incorecta. Cand apare depasirea, data existenta in acumulator poate fi foarte indepartata de valoarea corecta.

Pentru a intelege efectele depasirii, sa consideram adunarea numerelor in baza 10 intr-un sistem in care numerele nu pot fi mai mari de doi digiti. Daca adunam numerele 50, 45 si 20, rezultatul stocat in acumulator va fi 15, deoarece doi digiti nu sunt suficienti pentru a reprezenta corect rezultatul 115.

Chiar daca registrul acumulator nu intra in depasire, efectul poate aparea cand valorile acumulate sunt transferate memoriei, daca acumulatorul are biti de garda. Daca sunt folositi biti de garda, acumulatorul poate reprezenta numere mai lungi decat poate stoca o locatie de memorie. Acesta este cazul multor procesoare DSP, incluzind PineDSPCore (DSP Group) si DSP16xx (AT&T). Depasirea poate sa apara si daca un deplasator este folosit pentru a scala valoarea acumulatorului in vederea stocarii in memorie.

Exista doua modalitati obisnuite de tratare a depasirii. Prima tehnica este aceea de *a scala* cu grija toate rezultatele calculelor, eliminind astfel posibilitatea de aparitie a depasirii, indiferent de marimea datelor de la intrare. Totusi aceasta modalitate poate conduce la scalarea datelor la valori atat de mici incat nu poate fi mentinuta o fidelitate adecvata de reprezentare. O alternativa este folosirea *aritmeticii cu saturare* (saturation arithmetic). In aceasta aritmetica, un circuit special

detecteaza aparitia depasirii scalei si inlocuieste valoarea eronata de la iesire cu cel mai mare numar pozitiv care poate fi reprezentat (in cazul depasirii in directie pozitiva) sau cu cel mai mic numar negativ care poate fi reprezentat (in cazul depasirii in directie negativa). Rezultatul este desigur incorect, dar eroarea este mai mica decat daca functionarea continua fara saturare. Referindu-ne iarasi la calculatorul zecimal cu 2 digiti, rezultatul adunarii numerelor 50, 45, si 20 fara saturare este 15, situat departare de valoarea corecta de 115. Cu saturare, rezultatul este 99 (cel mai mare numar pozitiv care poate fi reprezentat cu 2 digiti), mai apropiat de valoarea corecta.

Pentru ca adesea nu este practic sau de dorit sa scalam semnalele pentru a elimina posibilitatea de aparitie a depasirii, aritmetica cu saturare este folositoare. Procesoarele DSP cu virgula fixa asigura hardware specializat pentru aritmetica cu saturare, astfel incat aceasta intervine automat (uneori sub controlul unui registru de mod) sau prin executarea unei instructiuni speciale. Unitatea hardware care implementeaza aritmetica cu saturare, este numita limitator de unii producatori.

- **Trunchierea si rotunjirea**

Asa cum am prezentat mai inainte, multiplicarea, acumularea, si alte operatii aritmetice tind sa creasca numarul de biti necesari pentru a reprezenta datele rezultatele fara pierderea preciziei. De la un punct insa, este necesara reducerea lungimii datelor rezultate (de exemplu, cand transferam continutul unui acumulator pe 36 biti intr-o locatie de memorie pe 16 biti). Cel mai simplu mod de a face acest lucru consta in descarcarea unui numar din cei mai putin semnificativi biti ai reprezentarii. Acesta operatie este numita *trunchiere*. De exemplu, pentru a trunchia o valoare pe 36 biti la 16 biti, cei mai putin semnificativi 20 de biti sunt descarcati, pastrind numai cei mai semnificativi 16 biti. Deoarece informatia continuta in cei 20 de biti descarcati este pierduta, este introdusa o eroare in reprezentarea datei. Se observa ca valoarea trunchiata este intotdeauna mai mica sau egala cu data originala. Cind datele sunt semnale trunchierea introduce un deplasament (bias sau offset) sau o componenta continua. O alta tehnica de reducere a numarului de biti este *rotunjirea*. Tehnicile de rotunjire reduc eroarea aritmetica si bias-ul introdus de aceasta reducere a preciziei. Ca si la saturare, unele procesoare executa automat rotunjirea cand rezultatele sunt transferate intre anumite registre si memorie (aceasta operatie poate fi activata printr-un registru de mod). Alte procesoare poseda o instructiune speciala pentru rotunjire.

Cea mai simpla modalitate de rotunjire este cea asa-numita tehnica de rotunjire la valoarea cea mai apropiata. Acesta este tipul conventional de rotunjire folosit de aritmetica curenta, si este tipul de rotunjire folosit de majoritatea procesoarelor DSP cu virgula fixa. Prin aceasta modalitate, numerele sunt rotunjite la cea mai apropiata valoare reprezentabila in formatul de iesire (cu precizie redusa); numerele care se afla exact la mijlocul distantei dintre doua valori de iesire consecutive sunt intotdeauna rotunjite la valoarea mai mare (pozitiva). Modul standard de implementare a

acestei metode consta in adunarea unei constante egala cu jumatate din valoarea celui mai putin semnificativ bit al cuvantului de iesire la valoarea care urmeaza sa fie rotunjita, si apoi trunchierea rezultatului la lungimea dorita. Unele procesoare poseda o instructiune speciala pentru a facilita rotunjirea prin preincarcarea acumulatorului cu o constanta potrivita. Fara o asemenea instructiune, rotunjirea poate fi realizata folosind o instructiune normala de incarcare sau de adunare a constantei in acumulator, inaintea trunchierii.

Faptul ca aceasta rotunjire modifica intotdeauna numerele care se afla exact la jumatatea distantei dintre doua valori consecutive de la iesire la valoarea mai mare dintre cele doua, introduce o asimetrie. Daca rotunjim un set de valori ale semnalului in acest mod, in medie mai multe valori sunt rotunjite in sus decat in jos. Aceasta inseamna ca operatia de rotunjire introduce o deplasare (bias) in semnal. Acest bias este insa mai mic decat bias-ul introdus de trunchiere. Chiar si asa, pentru unele aplicatii, cum ar fi filtrele IIR (filter cu raspuns infinit la impulsul unitate) sau filtrele de adaptare, acest mic bias poate creea probleme. Tehnica rotunjirii are insa avantajul de a fi usor de implementat.

O schema imbunatatita de rotunjire, care reduce bias-ul care apare la tehnica de rotujire descrisa anterior, este “rotunjirea convergenta”. Rotunjirea convergenta este putin mai complicata decat tehnica de rotunjire la valoarea cea mai apropiata. In cazul rotunjirii convergente, cand un numar care urmeaza sa fie rotunjit este situat exact la mijlocul dintre doua valori de iesire consecutive, poate fi rotunjit la o valoare mai mare sau mai mica. Directia rotunjirii depinde de valoarea bitului aflat in pozitia care va deveni cel mai putin semnificativ bit (LSB) al cuvantului de la iesire. Daca acest bit este zero, atunci numarul este rotunjit in jos (in directia negativa), daca acest bit este unu, numarul este rotunjit in sus. Rotunjirea convergenta este comparata cu tehnica de rotunjire la valoarea cea mai apropiata in Fig. 4.5.

Bitul care este folosit pentru a decide daca rotunjirea in cazul numerelor situate la mijlocul domeniului se face in sau in jos se presupune ca are probabilitati egale sa fie zero sau unu. Prin urmare mecanismul rotunjeste in sus in jumatate din valori si in jos in cealalta jumatate. Daca aceasta presupunere este valabila, rotunjirea convergenta evita bias-ul produs de rotunjirea la valoarea cea mai apropiata. Chiar daca este relativ simplu de implementat, mecanismul de rotunjire convergenta nu are suport hardware pentru majoritatea procesoarelor DSP cu vigula fixa. Familiile ADSP-21xx (Analog Devices) si DSP5600x (Motorola) sunt doua din cele care efectueaza rotunjire convergenta. Procesoarele care suporta rotunjirea convergenta pot de asemenea sa execute rotunjirea conventionala folosind tehnicile descrise mai sus.

Eroarea introdusa prin rotunjire si trunchiere in algoritmii DSP este deseori modelata ca zgomot aleator. Principala diferenta intre aceste trei tehnici de reducere a preciziei este bias-ul care se adauga la semnal. In unele aplicatii (in special in telecomunicatii), tehnica de cuantizare care trebuie folosita este specificata de un anumit standard tehnic.

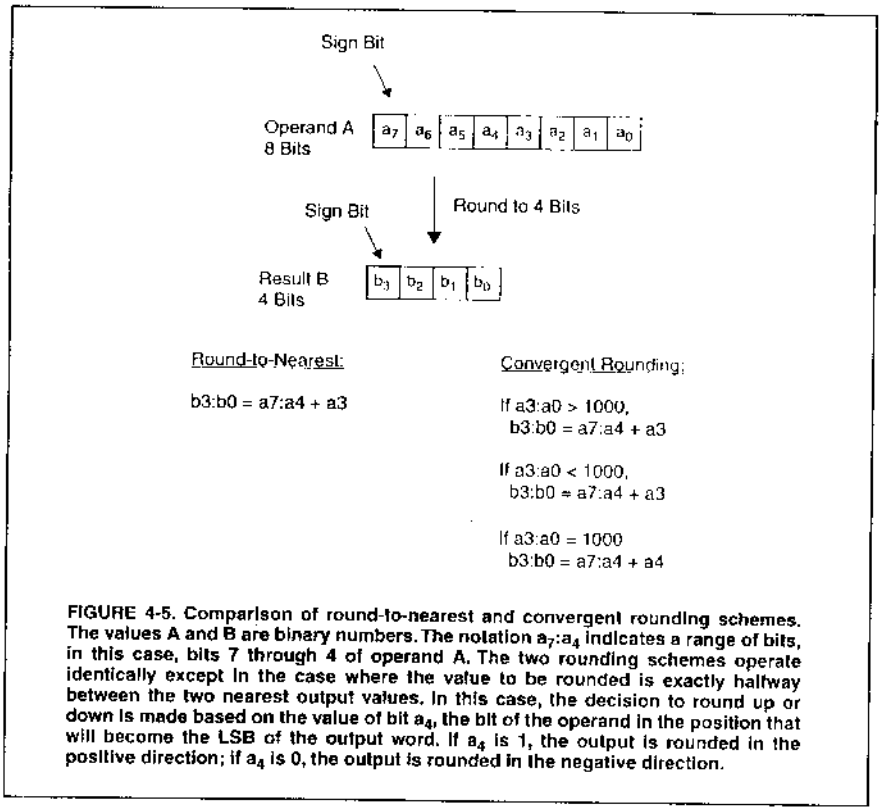


FIGURE 4-5. Comparison of round-to-nearest and convergent rounding schemes. The values A and B are binary numbers. The notation a_7:a_4 indicates a range of bits, in this case, bits 7 through 4 of operand A. The two rounding schemes operate identically except in the case where the value to be rounded is exactly halfway between the two nearest output values. In this case, the decision to round up or down is made based on the value of bit a_4, the bit of the operand in the position that will become the LSB of the output word. If a_4 is 1, the output is rounded in the positive direction; if a_4 is 0, the output is rounded in the negative direction.

- **Operanzii si registrii operand**

In majoritatea procesoarelor DSP cu vigula fixa, operanzii de pe calea de date sunt obtinuti de la un mic numar de registri operand sau de la acumulator. Inainte de a fi prelucrate datele trebuie sa fie transferate din memorie in registrii operand folosind instructiuni “move”. Celelalte instructiuni nu pot adresa direct operanzi din memorie. Un procesor care prelucreaza datele numai din registrii operand este considerat ca avand o arhitectura “load-store” (incarca-stocheaza). Registrii pentru operanzi sunt accesati folosind modul de adresare directa la registru (register-direct). (detalii privind modurile de adresare in Capitolul 6.)

In cazul unor procesoare (de exemplu, TMS320C2x/5x -Texas Instruments si Pine DSPCore - DSP Group), operanzii pot fi preluati direct din memorie, folosind adresarea directa la memorie sau adresarea indirecta cu registru.

4.2 Calea de date la procesoarele cu virgula mobila

Calea de date la procesoarele DSP cu virgula mobila este similara cu cea de la procesoarele DSP cu virgula fixa, cu unele diferente. In acesta sectiune, vom evidentia acele aspecte care diferentiaza calea de date la procesoarele DSP cu virgula mobila de cea de la procesoarele cu

virgula fixa. Fig. 4.6 prezinta o cale de date tipica, folosita la procesorul cu virgula mobila pe 32 de biti, DSP3210 (AT&T).

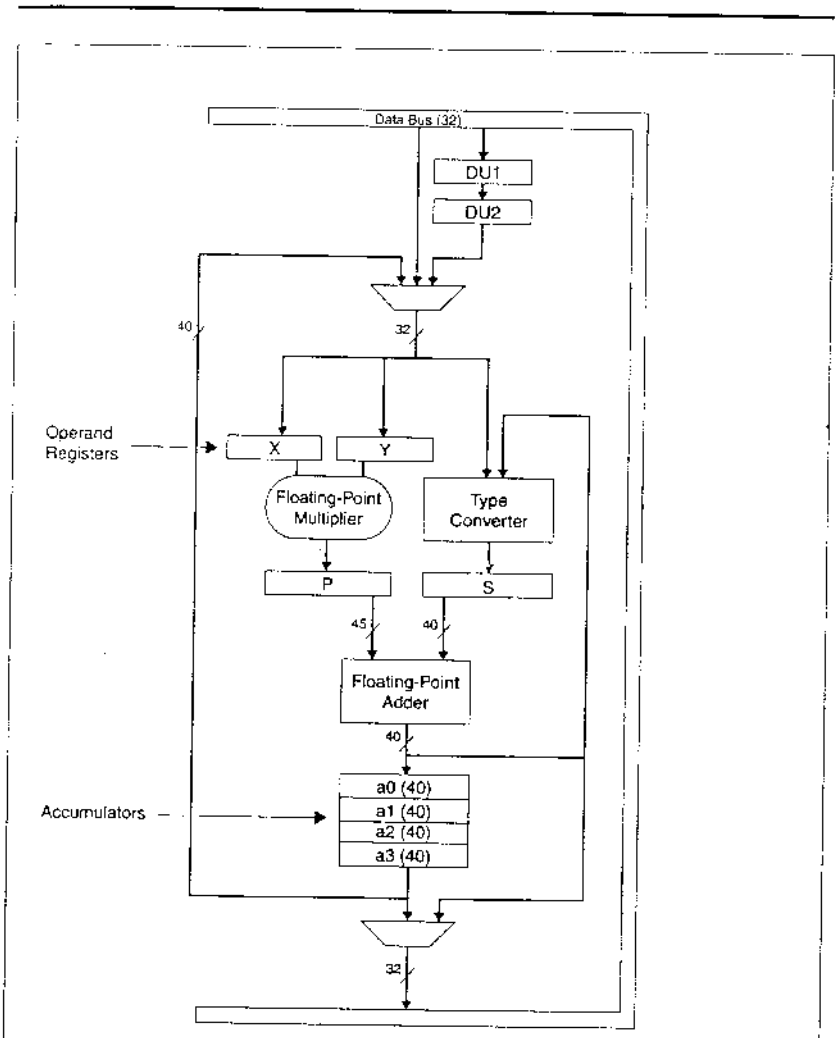


FIGURE 4-6. A typical floating-point DSP processor data path (from the AT&T DSP3210).

La majoritatea procesoarelor DSP cu virgula mobila, principala cale de date este capabila sa faca calcule atat in virgula mobila cat si in virgula fixa, dar poate sa execute un singur tip de operatie intr-un anumit ciclu de instructiune. Este cazul procesoarelor DSP de la Texas Instruments, Analog Devices, Motorola.

Unele procesoare cu virgula mobila furnizeaza doua cai de date: una pentru operatii in virgula mobila si una (cu mai putine facilitati) pentru operatiile in virgula fixa. De exemplu, la procesorul DSP32xx (Motorola), unitatea aritmetica de control efectueaza atat calculul adreselor cat si operatii aritmetice cu intregi, iar unitatea arimetica pentru date (data arithmetic unit) executa calculele in virgula mobila. Procesoarele care au cai de date diferite pentru virgula mobila si virgula fixa, nu poseda multiplicator pe calea de date pentru operatii in virgula fixa.

- **Multiplicatorul**

Multiplicatoarele DSP cu virgula mobila accepta doi operanzi cu virgula mobila de marime nativa (de obicei 32 biti). Spre deosebire de multiplicatoarele procesoarelor DSP cu virgula fixa, multiplicatoarele pentru operatii in virgula mobila nu produc la iesire un cuvant suficient de mare pentru a evita pierderea preciziei. De exemplu, cand sunt inmultite doua date in virgula mobila cu precizie simpla IEEE-754, fiecare data de intrare are largimea efectiva a mantisei de 24 biti. Pentru a mentine intrega precizie, data de iesire ar necesita o mantisa de largime efectiva de 48 biti. Majoritatea DSP-urilor cu virgula mobila nu admit cuvinte de acesta lungime. Totusi formatul datelor de iesire la multiplicatoarele in virgula mobila este in general ceva mai lung decat formatul de la intrare, asigurind 8 sau 12 biti in plus pentru reprezentarea mantisei.

- **ALU**

Unitatea ALU a procesorului DSP cu virgula mobila executa operatii simple: adunari, scaderi, si alte operatii aritmetice cum ar fi calculul valoarii absolute, negari, calculul valorii minime sau/si a valorii maxime. Alte ALU executa si operatii specializate, lucru exemplificat in Tabelul 4.1.

Tabelul 4.1

Operation	Description	Example Processors
Reciprocal seed	Provides an estimate of the reciprocal of a value (i.e., $1/x$). The estimate can be used as the starting point, or *seed*, for an iterative algorithm that computes the precise value of the reciprocal.	ADSP-210xx DSP96002 DSP32xx
Square root reciprocal seed	Provides an estimate of the reciprocal of the square root of a value. The estimate can be used as the starting point, or *seed*, for an iterative algorithm that computes the precise value of the reciprocal of the square root.	ADSP-210xx DSP96002
Add and subtract	Simultaneously computes the sum and the difference of two data values and deposits the results into two registers.	ADSP-210xx DSP96002
IEEE format conversion	Converts between the processor's internal floating-point data format and IEEE-754 floating-point format.	DSP32xx TMS320C4x
Integer floating-point conversion	Converts between internal floating-point and internal integer formats.	DSP32xx TMS320C3x TMS320C4x

Procesoarele cu virgula mobila folosesc ALU pentru a executa adunarea in operatiile de multiplicare-acumulare. In plus fata de operatia de multiplicare-acumulare, unele procesoare (cum ar fi DSP32C - AT&T) executa o operatie de multiplicare-adunare. Operatia de multiplicare-

adunare este diferita de cea de multiplicare-acumulare prin faptul ca rezultatul este scris intr-un acumulator diferit de cel care furnizeaza data care este adunata cu produsul.

Operatiile logice pe bit cum ar fi operatiile logice SI, SAU, NEGARE nu sunt semnificative cand sunt aplicate datelor cu virgula mobila si in general nu sunt executate de catre ALU destinat numai operatiilor in virgula mobila (floating-point-only ALU).

- **Overflow, underflow si alte exceptii**

La aritmetica in virgula mobila, depasirea de scala (overflow) si alte situatii asemanatoare sunt numite exceptii. Exceptiile sunt situatii neobisnuite care pot cauza rezultate aritmetice eronate. Procesorul semnalizeaza aparitia unei exceptii setind un bit intr-un registru de stare sau generind o intrerupere (situatie optionala la unele procesoare) care antreneaza executia unei rutine de prelucrare a exceptiei, furnizata de programator.

Depasirea este o situatie mai putin grava la procesoarele DSP cu virgula mobila decat pentru cele cu virgula fixa datorita gamei dinamice mai mari asigurata de formatele cu virgula mobila (vedeti capitolul 3 pentru detalii). Totusi pentru unele aplicatii depasirea ramane o problema reala. Majoritatea procesoarelor DSP cu virgula mobila inregistreaza aparitia depasirii setind un bit de stare si limitind automat rezultatul operatiei care a cauzat depasirea (setindu-l la cea mai pozitiva sau la cea mai negativa valoare reprezentabila la iesire).

Alta situatie exceptionala intalnita la proceosarele cu virgula mobila este cea de "underflow". Underflow apare cand rezultatul unei operatii aritmetice este prea mic pentru a fi reprezentat. Aceasta se intampla, de exemplu, cand inmultim doua numere subunitare mici. In acest caz, actiunea obisnuita pentru mai multe procesoare este setarea rezultatului la zero, iar pentru a indica aparitia underflow-lui este setat un bit intr-un registru de stare.

Si alte situatii exceptionale sunt de asemenea semnalate; de exemplu, impartirea la zero cauzeaza o exceptie pentru procesoarele care executa operatia "reciprocal seed".

- **Rotunjirea**

Ca si la aritmetica cu virgula fixa, multiplicarea, adunarea si alte operatii aritmetice cu virgula mobila tind sa creasca numarul de biti ai rezultatelor. Majoritatea procesoarelor DSP cu virgula mobila rezolva problema cresterii preciziei rotunjind automat rezultatele aritmetice la un format intermediar de 40÷44 biti. Acest format intermediar este folosit pentru calcule care au loc in interiorul caii datelor. Cand datele sunt transferate in memorie, ele pot fi scrise fie ca valori de precizie extinsa (ceea ce necesita mai multe locatii de memorie si mai multe operatii "move" de la registru la memorie), fie pot fi rotunjite la formatul nativ de precizie redusa, in general 32 de biti.

Ca si procesoarele cu virgula fixa, majoritatea procesoarelor cu virgula mobila asigura cel mai simplu mod de rotunjire, rotunjirea la nivelul cel mai apropiat. Unele procesoare ofera doua sau

mai multe optiuni pentru rotunjirea datelor cu virgula mobila. DSP96002 de la Motorola este unic prin aceea ca asigura toate modurile de rotunjire specificate de standardul IEEE 754 pentru valorile de precizie simpla: rotunjirea convergenta, rotunjirea catre plus infinit, rotunjirea catre zero, si rotunjirea catre minus infinit.

- **Registrii acumulator**

In general, procesoarele cu virgula mobila au registri mai multi si de capacitate mai mare decat cele cu virgula fixa. In unele procesoare cu virgula mobila (de exemplu, DSP3210 de la AT&T) un mic numar de registri sunt special proiectati pentru a fi folositi ca acumulatori. Alte procesoare ofera o serie de registri de uz general, un subset al celor ce pot primi rezultatele operatiei de multiplicare-acumulare sau ale altor operatii aritmetice. Acesta este cazul procesorului TMS320C3x - Texas Instruments, care are opt registri pe 40 biti si opt pe 32 biti, care pot primi rezultatele operatiilor aritmetice.

- **Deplasatorul**

Ca si in cazul aritmeticii cu virgula fixa, o operatie de multiplicare-acumulare cu virgula mobila poate sa duca la cresterea numarului de biti a rezultatului. La aritmetica cu virgula mobila hardware-ul scaleaza automat rezultatele pentru a conserva maximul de precizie posibil. Acesta este principalul avantaj al aritmeticii in virgula mobila, asa cum este descris in detaliu in Capitolul 3. Calea de date incorporeaza un deplasator pentru a executa scalarea, dar deplasatorul fie nu este vizibil, fie nu este explicit controlabil de catre programator la operatiile in virgula mobila. La procesoarele cu o singura cale a datelor care executa atat operatiile in virgula mobila cat si pe cele in virgula fixa, deplasatorul poate fi explicit controlat de programator pentru deplasarea datelor reprezentate in virgula fixa.

- **Operanzii si registrii operand**

Majoritatea procesoarelor cu virgula mobila folosesc operanzi pentru caile de date provenind de la registri operand sau de la acumulatoare. Pentru aceste date este folosita adresarea directa la registru. (In Capitolul 6 este facuta o discutie detaliata asupra modurilor de adresare.) Datele sunt incarcate din memorie in registrii operand cu instructii explicite “move”. (Unele procesoare pot executa aceste instructiuni “move” in paralel cu instructiuni aritmetice- -vezi capitolul 7 pentru mai multe detalii asupra paralelismului instructiilor.)

Cateva procesoare (de exempalu, DSP32xx - AT&T) transfera operanzi pe calea de date direct din memorie folosind adresarea indirecta cu registru.

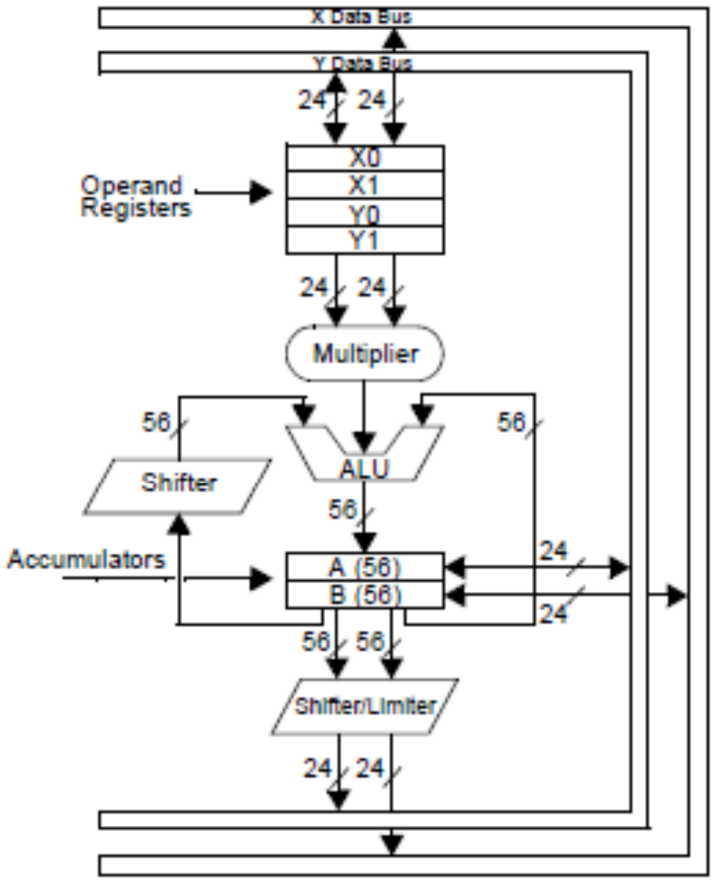


FIGURE 1. A representative conventional fixed-point DSP processor data path (from the Motorola DSP560xx, a 24-bit, fixed-point processor family).